



Figure 2: Follow the TCP stream

Box 2: Format of the RSA private key

```
-----BEGIN RSA PRIVATE KEY-----
MAIEpAIBAAKCAQEAxcVEgXm4nNlSeFneHmWIOO/gx-
aQnhZ/Fx84RmWW9DXRUhU6P
YtdMIZ03C4z5W6t7JyLpTsCJ1egWE5CSlyRY37ncs
qJsu8WQ6CsIfPIEERmQVh4K92bryK
.
.
WGcsdfrG/rWcpYpInfsGbu0Qd25ZwirepLOcBobX
UMXaQ1JoG0AokLEoVECCtTnDaKh
HEJ4yX4GsdHfHseSdsf9sdHf9In0DBRjoQ3ttJdApo
MwqCMK4CCfB2F4eQcwhg==
-----END RSA PRIVATE KEY-----
```

G. Start the browser and open <https://vmwareesxi>

- Right-click any packet and look at what's shown under 'Follow TCP stream' – this will show the encrypted https traffic as seen in Figure 2.
- Use 'Follow SSL stream' to view the traffic in the decrypted format as depicted in Figure 3.

If all the steps are followed correctly, the captured traffic will be seen in a decrypted format.

Protecting against https RSA decryption

This decryption method is successful only if RSA is used to encrypt and decrypt the symmetric key – which is sent across from the client to the server via the network (in encrypted form). So is there a way out? Yes, definitely.

The Diffie Hellman (DH) key exchange method plays a vital role in ensuring the security of https. DH creates a shared secret between two systems without actually transmitting the 'encryption key' across the wire. However, DH is only a key exchange method; it does not authenticate the server nor does it protect the data.

By combining RSA, which provides server authentication and DH (which enables the creation of a shared encryption key at the client and the server ends), the result is excellent protection against https decryption even if a private key is available.

RSA and DH in combination are used by various open source distros such as IPCop and OpenFile, thus making sure that a compromised private key is not sufficient for successful decryption.

Diagnosing decryption related issues

1. Make sure that you capture all the traffic right from the first packet in order to enable decryption. A point to ponder over is that the encryption key is transmitted in encrypted format between client and server before the actual data transfer. If you miss this traffic, the key will not be available and further decryption will not be possible.
2. Use Wireshark decryption logs, which can be enabled by specifying the log file name under *Edit – Preferences – Protocols – SSL – (Pre)-Master-Secret*. This file can then be checked to find the decryption status. Please see Box 3 and Box 4 for unsuccessful and successful decryption logs.

Box 3: Unsuccessful decryption

```
dissect_ssl3_enter frame #23 (first time) conversation
= 0x7fda095a1d18, ssl_session = 0x7fda095a21c8
record: offset = 0, reported_length_remaining = 1091
dissect_ssl3_record found version 0x0301 -> state
0x11
dissect_ssl3_record: content_type 22
decrypt_ssl3_record: app_data len 49, ssl state 0x11
packet from_server: is from server - TRUE
decrypt_ssl3_record: using server decoder
decrypt_ssl3_record: no decoder available
dissect_ssl3_handshake iteration 1 type 2 offset 5
length 45 bytes, remaining 54
dissect_ssl3_hnd_hello_common found SERVER
RANDOM -> state 0x13
ssl_restore_session can't find stored session
dissect_ssl3_hnd_srv_hello found CIPHER 0x0035 ->
state 0x17
dissect_ssl3_hnd_srv_hello trying to generate keys
ssl_generate_keying_material not enough data to gen-
erate key (0x17 required 0x37 or 0x57)
Please use this text in bold or as a footer for Box 3.
```

Box 4: Successful decryption

```
ssl_association_remove removing TCP 443 - http handle
0x7fd2ef884150
Private key imported: KeyID bc:8f2cfc:c2:54:67:f7:cc:0a
:1c:34:21:b3:80:9e:...
ssl_init IPV4 addr '192.168.51.2' (192.168.51.2)
port '443' filename '/home/rajesh/Desktop/ruik.key'
password(only for p12 file) ''
ssl_init private key file '/home/rajesh/Desktop/ruik.key' suc-
cessfully loaded.
association_add TCP port 443 protocol http handle
0x7fd2ef884150
```

In this article, I have tried to describe various encryption